Certificate of Integrity

Official release record for **Student Timer Clock version 0.3** (Windows x64)

Record number	STC-REL-0.3-20261106-5156EB1E
Product	Student Timer Clock
Version	0.3 (file version 0.3.0.0)
Platform	Microsoft Windows 10 / 11, x64 · .NET 8 (self-contained in the installer)
Date of issue	Friday, 06 November 2026, 03:08:17 UTC
Signature status	VALID - all executables carry a valid SHA-256 Authenticode signature with RFC 3161 timestamp

SHA-256 of the core executable (StudentTimerClock.exe, as installed by the setup program)
`5156EB1E 794AA2A9 CA609C28 FFC71DC4 52E445D9 BDE249DC 3034D356 3CDA977B`

1. Release artefacts and SHA-256 hashes

The SHA-256 values below identify the exact, unmodified files of this release. Any change to a file - even a single bit - produces a completely different value.

Role	**Core executable (installed)**
File	StudentTimerClock.exe
Size	62,495,728 bytes (59.60 MB)
SHA-256	`5156EB1E 794AA2A9 CA609C28 FFC71DC4 52E445D9 BDE249DC 3034D356 3CDA977B`
Notes	Self-contained single-file build; contains all program code and the .NET runtime. Installed to %LOCALAPPDATA%\Programs\StudentTimerClock.

Role	**Installer**
File	StudentTimerClock version 0.3.exe
Size	56,351,624 bytes (53.74 MB)
SHA-256	`33AC8A74 1A4B5402 AD386652 94BBF7D6 0ED463D7 E627E0BB FD7E7E26 B6FD5717`
Notes	NSIS setup program (installer and uninstaller are signed).

Role	**Portable executable**
File	StudentTimerClock.exe (portable)
Size	1,065,800 bytes (1.02 MB)
SHA-256	`78C4CF14 AD86D2BC C73B6CBE 93877BB2 5D847EC4 A8BB8C02 859B3E48 F403A3BA`
Notes	Framework-dependent single-file build inside the portable ZIP. Note: the .NET launcher of framework-dependent apps loads hostfxr.dll from the program folder before the program's own integrity checks run - keep the portable folder write-protected; the installed edition is not affected.

Role	**Portable package**
File	StudentTimerClock version 0.3 (portable, needs NET 8).zip
Size	541,758 bytes
SHA-256	`9A06CF43 551B3DF4 878BA944 804CA219 D22EE1E7 0D5FDA30 805D28F1 6C5F9DEE`
Notes	ZIP archive: portable executable + signed integrity manifest.

Role	**Integrity manifest**
File	integrity.json (installed)
Size	458 bytes
SHA-256	`9C460567 A38538F6 A2E2AA85 98E740DB 0DD5DDBC C702B7A2 EA6F3037 F1F015A1`
Notes	Signed list of SHA-256 hashes checked by the program at runtime (signature: integrity.sig).
Role	**Root CA certificate**
File	Student Timer Clock Root CA.cer
Size	1,461 bytes
SHA-256	`F4357B03 268B927B 4B09C38E 35AD58CE 1625B764 E65C8619 99A2E027 E47DEA3B`
Notes	Public certificate of the issuing CA.
Role	**Signing certificate**
File	Student Timer Clock Code Signing.cer
Size	1,406 bytes
SHA-256	`9FAFAC5D C3576A11 C7E73617 7A84212D 56EC2BCE F76FE5E6 1D31FFC0 73DE7FDD`
Notes	Public code-signing certificate.

2. Code-signing certificate

Subject	CN=Student Timer Clock Code Signing,OU=Release Engineering,O=Student Timer Clock,L=Cairo,C=EG
Issuer	CN=Student Timer Clock Root CA,OU=Release Engineering,O=Student Timer Clock,C=EG
Serial number	`6E4948A4CB14F78FFF17CAAB8AB0BD7FB38B7FD7`
Valid from	06 November 2026, 02:33:48 UTC
Valid until	05 November 2031, 02:33:48 UTC
Public key	RSA 3072-bit
Signature algorithm	SHA256 with RSA
Key usage	Digital Signature
Extended key usage	Code Signing (1.3.6.1.5.5.7.3.3)
Basic constraints	End entity (not a CA)
Thumbprint (SHA-1)	`DDAA717F D6468B4F 9BF2E736 077AE9B5 4BA2B2CD`
Fingerprint (SHA-256)	`9FAFAC5D C3576A11 C7E73617 7A84212D 56EC2BCE F76FE5E6 1D31FFC0 73DE7FDD` `= certificate pinned inside the executable`

Issuing certificate authority

Subject	CN=Student Timer Clock Root CA,OU=Release Engineering,O=Student Timer Clock,C=EG
Type	Self-signed root certificate (private CA of the project)
Serial number	`6CC2A3DB24BB72CB4730D17D87E2194D9A9DA303`
Valid	06 November 2026, 02:33:48 UTC - 03 November 2036, 02:33:48 UTC
Public key	RSA 4096-bit
Thumbprint (SHA-1)	`02F998CD D8E1804C 8BEC0FC0 07561713 3689B90A`
Fingerprint (SHA-256)	`F4357B03 268B927B 4B09C38E 35AD58CE 1625B764 E65C8619 99A2E027 E47DEA3B`

3. Digital signatures (Authenticode)

File	**Core executable (installed)**
Result	**Signature valid** - digest recomputed and matched; CMS signature verified against the certificate chain
Digest algorithm	SHA256
Signed image digest	`6C9722F2 E3FEB82F 3DEB3809 25FA9E09 9008D7BC B69F5D91 48F15176 46835AE2`
Signing time	Nov 6 03:02:56 2026 GMT
Timestamp (RFC 3161)	Nov 6 03:02:56 2026 GMT · /C=US/O=DigiCert, Inc./CN=DigiCert Trusted G4 TimeStamping RSA4096 SHA256 2025 CA1

File	**Installer**
Result	**Signature valid** - digest recomputed and matched; CMS signature verified against the certificate chain
Digest algorithm	SHA256
Signed image digest	`08F8C0EE 15E2B710 FE5BB0B2 37046FB1 A2469784 29147B97 DD5E946E BB1EB67F`
Signing time	Nov 6 03:03:36 2026 GMT
Timestamp (RFC 3161)	Nov 6 03:03:36 2026 GMT · /C=US/O=DigiCert, Inc./CN=DigiCert Trusted G4 TimeStamping RSA4096 SHA256 2025 CA1

File	**Portable executable**
Result	**Signature valid** - digest recomputed and matched; CMS signature verified against the certificate chain
Digest algorithm	SHA256
Signed image digest	`AAB0663C 8B4CDF37 ACE4B0F0 063C839B C1392C63 4D1EC1C7 B8142D38 42A26821`
Signing time	Nov 6 03:02:56 2026 GMT
Timestamp (RFC 3161)	Nov 6 03:02:56 2026 GMT · /C=US/O=DigiCert, Inc./CN=DigiCert Trusted G4 TimeStamping RSA4096 SHA256 2025 CA1

The signed image digest is the Authenticode hash defined by Microsoft (the whole file except the checksum, certificate-table entry and signature). The trusted timestamp proves the signature was made while the certificate was valid, so it stays valid after the certificate expires.

4. Built-in tamper protection

At every start, before any window opens	Self-verification of the running executable: Authenticode digest recomputed, CMS signature verified, signer certificate compared with the pinned SHA-256 fingerprint above; full-file SHA-256 compared with the signed integrity manifest; Windows WinVerifyTrust verdict.
Every 5 minutes	The same signature and hash checks are repeated; the program folder is scanned for files that are not in the signed manifest (DLL planting / side-loading).
Every 10 seconds	Loaded-module scan (every DLL must come from Windows / Program Files, be listed in the signed manifest, or carry a valid trusted signature); debugger detection; .NET profiler / startup-hook injection detection.
Process hardening	Current directory removed from the DLL search path; DLL search restricted to System32 and the program folder; AppInit/global-hook/LSP extension DLLs blocked; DLLs from network shares and low-integrity locations blocked; .NET startup hooks disabled at build time.
On any violation	The event is written to security.log, all functionality is frozen, the user is informed and the process is terminated. User data is never modified or deleted.
Input validation	Strict allow-list validation of every value that reaches a process start, shell, mailto:, keystroke or file sink; CSV formula-injection protection; size, depth and zip-bomb limits for data and backups; actions from backups or data files changed outside the app are quarantined until confirmed by the user; data file sealed with a DPAPI-protected HMAC-SHA256.

5. How to verify this release yourself

Windows PowerShell - compare the result with section 1:

```
Get-FileHash "$env:LOCALAPPDATA\Programs\StudentTimerClock\StudentTimerClock.exe" -Algorithm SHA256
Get-FileHash ".\StudentTimerClock version 0.3.exe" -Algorithm SHA256
Get-AuthenticodeSignature ".\StudentTimerClock version 0.3.exe" | Format-List Status, SignerCertificate,
TimeStamperCertificate
```

Command prompt:

```
certutil -hashfile "StudentTimerClock version 0.3.exe" SHA256
signtool verify /pa /v "StudentTimerClock version 0.3.exe" (Windows SDK)
```

Explorer: right-click the file → Properties → **Digital Signatures** → Details → View Certificate; the certificate thumbprint must equal the SHA-1 thumbprint in section 2. The file **SHA256SUMS.txt** in the release folder carries a detached CMS signature (SHA256SUMS.txt.p7s) made with the same key; this document is accompanied by a detached signature as well (*…Certificate of Integrity.pdf.p7s*).

6. Important notice on trust

The code-signing certificate was issued by the project's own private certificate authority (self-signed root), not by a publicly trusted commercial CA. The signatures are cryptographically valid and prove that the files are unchanged since they were signed by the holder of the project's private key. However, Windows does not know this CA: Explorer and PowerShell report the root as *not trusted*, and SmartScreen may still show *Unknown publisher*. An organisation can establish trust by deploying *Student Timer Clock Root CA.cer* to the *Trusted Root Certification Authorities* and *Trusted Publishers* stores (e.g. via Group Policy). For public distribution, re-sign with an OV/EV certificate from a public CA - the build pipeline accepts any PFX (SIGN_PFX) and re-pins the certificate automatically.

This record is generated automatically by the release build pipeline from the actual release files. It is a technical attestation of file identity and signature status; it is not a third-party security audit, a legal certification, or a guarantee that the software is free of defects. Client-side tamper protection raises the effort needed to modify the program considerably, but no software running on a computer controlled by an attacker with administrative rights can be made impossible to modify.

Attestation	**Signed by**
The files listed in section 1 were built from source, digitally signed and verified by the Student Timer Clock release pipeline on 06 November 2026 at 03:08 UTC. The values in this record were computed from those files.	Student Timer Clock Code Signing `DDAA 717F D646 8B4F 9BF2 E736 077A E9B5 4BA2 B2CD` Record STC-REL-0.3-20261106-5156EB1E